

BÀI TẬP ÁP DỤNG DFS, BFS TRÊN MẢNG 2 CHIỀU (LƯỚI Ô VUÔNG)

1. Một số bài toán áp dụng

Bài 1: Tần số phát sóng (Trích đề thi chọn HSG lớp 12 tỉnh Hà Tĩnh năm học 2013 - 2014)

Người ta phân hoạch một vùng đất hình chữ nhật thành $M \times N$ ô vuông bởi các đường kẻ song song với các đường biên của vùng đất đó. Vị trí của mỗi mảnh đất ô vuông được xác định bởi cặp số (x, y) trong đó x là số thứ tự dòng tính từ trên xuống dưới, y là số thứ tự cột tính từ trái sang phải của mảnh đất ô vuông đó khi ta ở một tư thế nhìn từ một mặt phẳng song song phía trên mặt phẳng chứa vùng đất. Tại mỗi vị trí ô vuông của mỗi vùng đất được đặt một máy phát sóng vô tuyến điện tử. Mỗi máy có thể phát ra các loại sóng có dải tần số A hoặc B; tại mỗi thời điểm phát sóng chỉ phát ra mỗi loại sóng có một dải tần số nào đó mà thôi. Ta có thể di chuyển từ vị trí máy phát sóng này đến vị trí máy phát sóng khác nếu hai máy phát sóng đó nằm trên hai mảnh đất ô vuông có chung cạnh. Một vùng máy phát sóng là tập hợp các mảnh đất ô vuông kề cách với nhau và trên đó đặt các máy phát sóng phát ra loại sóng có cùng dải tần số. Việc xác định hai máy phát sóng nào đó nằm trên hai mảnh đất ô vuông có cùng một vùng phát sóng tại một thời điểm phát sóng nào đó hay không có ý nghĩa trong việc lên kế hoạch phát sóng trong lần phát sóng kế tiếp.

Cho trước vị trí của hai mảnh đất ô vuông, hãy xác định hai mảnh đất ô vuông đó có cùng một vùng phát sóng tại một thời điểm phát sóng nào đó hay không.

Dữ liệu vào là tệp văn bản TSPS.INP có cấu trúc:

- Dòng đầu tiên ghi hai số N, M ($1 \leq M \leq 100, 1 \leq N \leq 100$).
- N dòng tiếp theo, mỗi dòng ghi N ký tự A hoặc B nếu máy phát sóng đặt trên mảnh đất ô vuông tương ứng phát loại sóng có dải tần A hoặc B trong mỗi lần phát sóng.
- Các dòng tiếp theo, mỗi dòng ghi 4 số mô tả vị trí của hai mảnh đất ô vuông mà ta cần kiểm tra xem chúng có cùng một vùng phát sóng hay không; 2 số đầu là thứ tự dòng và thứ tự cột của mảnh đất ô vuông thứ nhất; 2 số sau là thứ tự dòng và thứ tự cột của mảnh đất ô vuông thứ hai.
- Các ký tự trên cùng một dòng được ghi liên tiếp nhau. Các số trên cùng một dòng được ghi cách nhau ít nhất một ký tự trống.

Dữ liệu ra là tệp TSPS.OUT có số dòng là số cặp mảnh đất ô vuông mà ta cần kiểm tra xem chúng có cùng một vùng phát sóng hay không? Mỗi dòng ghi một số là số 1 hoặc số 0 nếu cặp mảnh đất ô vuông tương ứng cùng một vùng phát sóng hoặc không.

Ví dụ:

TSPS.INP	TSPS.OUT
4 6	1
AABBAA	0
BAABBA	
BBAABB	
AABBBB	
1 2 3 4	
2 3 1 6	

Ý tưởng thuật toán.

- Vì bài toán này có cặp ô vuông cần kiểm tra có cùng vùng phát sóng hay không. Gọi hàm DFS hoặc BFS xác định các vùng liên thông (vùng phát sóng) và đánh dấu các vùng phát sóng để biết ô nào thuộc vùng phát sóng nào. Gọi hàm DFS hoặc BFS bắt đầu từ ô $(1, 1)$ và đánh dấu các ô thuộc vùng

phát sóng 1, sau đó kiểm tra mảng đánh dấu nếu có ô (i,j) nào đó chưa được đánh dấu thì tiếp tục gọi DFS hoặc BFS và đánh dấu các ô thuộc vùng phát sóng 2. Quá trình này tiếp tục cho đến tất cả các ô ở mảng đánh dấu được đánh dấu hết.

- Kiểm tra cặp ô vuông (x1,y1) và (x2,y2) có cùng vùng phát sóng hay không bằng cách kiểm tra giá trị mảng đánh dấu, nếu $DD[x1][y1]=DD[x2][y2]$ thì cặp ô vuông (x1, y1) và (x2, y2) cùng vùng phát sóng ngược lại thì không.

Chương trình tham khảo

```
#include <bits/stdc++.h>
using namespace std;
int n,m,dem=0;
char a[2003][2003];
int DD[2003][2003];
int dx[4]= {0, -1, 0 ,1};
int dy[4]= {-1, 0, 1, 0};
//=====
void nhap()
{
    freopen("TSPS.INP","r",stdin);
    freopen("TSPS.OUT","w",stdout);
    cin>>n>>m;
    for(int i=1;i<=n;i++)
        for(int j=1; j<=m;j++)
            cin>>a[i][j];
    memset(DD,0,sizeof(DD));
}
//=====
void DFS(int i, int j) {
    DD[i][j] = dem ;
    for(int k=0;k<4; k++) {
        int i1=i+dx[k] , j1= j + dy[k] ;
        if(i1>=1&& i1<=n && j1>=1 && j1 <=m && !DD[i1][j1]
        && a[i1][j1]==a[i][j]){
            DFS(i1,j1);
        }
    }
}
//=====
void xuly(){
    int x1, y1, x2, y2;
    for(int i=1;i<=n;i++)
        for(int j=1;j<=m;j++)
            if(!DD[i][j])
            {
                dem++;
                DFS(i,j);
            }
    while(cin>>x1>>y1>>x2>>y2)
    {
```

```

        if (DD[x1][y1]==DD[x2][y2]) cout<<1<<endl;
        else cout<<0<<endl;;
    }
}
//=====
int main()
{
    nhap(); xuly();
    return 0;
}

```

Bài 2: Đất quy hoạch *(Trích Đề thi HSG lớp 12 năm 2020-2021, tỉnh Hà Tĩnh)*

Công ty X muốn tìm mua một mảnh đất rộng để xây nhà máy, đại diện công ty X đến công ty mua bán bất động sản Y và được công ty Y cho xem bản đồ quy hoạch khu đất mà công ty Y đang muốn bán.

Trên bản đồ quy hoạch của khu đất hình chữ nhật, người ta chia bản đồ thành $m \times n$ ô vuông. Mỗi ô vuông (i,j) ghi một số nguyên dương a_{ij} ($1 \leq a_{ij} \leq 5$; $0 < i \leq m$; $0 < j \leq n$) là độ cao trung bình của ô đất so với mặt nước biển. Một mảnh đất là tập hợp tất cả các ô có cùng giá trị và có chung cạnh. Diện tích của mảnh đất là số lượng các ô thuộc mảnh đất đó.

Yêu cầu: Cho hai số nguyên dương m, n ($0 < m, n \leq 1000$) và bảng giá trị a_{ij} hãy:

- Đếm số mảnh đất của khu đất.
- Tìm mảnh đất có diện tích lớn nhất.

Dữ liệu: Vào từ file văn bản PLAN.INP gồm:

- Dòng đầu ghi hai số nguyên dương m, n cách nhau bởi dấu cách.
- m dòng tiếp theo mỗi dòng ghi n số nguyên a_{ij} , các giá trị trên cùng một dòng được ghi liên nhau.

Kết quả: Ghi ra file văn bản PLAN.OUT trên 2 dòng:

- Dòng đầu tiên ghi số nguyên k là số mảnh đất của khu đất.
- Dòng thứ hai ghi diện tích lớn nhất của mảnh đất.

Ràng buộc:

- Có 30% số test ứng với 30% số điểm của bài thỏa mãn điều kiện: $1 \leq a_{ij} \leq 2$, $n \leq 10$;

- Có 30% số test ứng với 30% số điểm của bài thỏa mãn điều kiện: $n \leq 20$;
- Có 30% số test ứng với 30% số điểm của bài không có ràng buộc gì thêm.

Ví dụ:

PLAN.INP	PLAN.OUT
3 5	4
11222	7
12212	
31112	

Ý tưởng thuật toán

- Bài toán này chính là bài toán đếm số thành phần liên thông trên lưới ô vuông và tìm vùng liên thông lớn nhất.
- Dùng biến d để đếm số ô vuông trong mỗi vùng liên thông, biến d tăng 1 khi loang sang ô kề cạnh trong hàm DFS hoặc BFS.
- Dùng biến Max để lưu giá trị vùng liên thông lớn nhất. Biến dem để đếm số vùng liên thông.
- Duyệt tất cả các ô của mảng đánh dấu, giả sử ô (i,j) chưa được đánh dấu ta gọi hàm DFS(i,j) hoặc BFS(i,j). Mỗi lần gọi hàm DFS(i,j) hoặc BFS(i,j) thì biến dem tăng 1, sau khi lời gọi hàm kết thúc so sánh Max với d nếu $d > \text{Max}$ thì $\text{Max} = d$
- Đưa kết quả dem, và Max ra.

Chương trình tham khảo :

```
#include <bits/stdc++.h>
using namespace std;
int n,m,x,y,d,dem=0, Max=0;
char a[1003][1003];
bool DD[1003][1003];
int dx[4]= {0, -1, 0 ,1};
int dy[4]= {-1, 0, 1, 0};
//=====
void nhap()
{
    freopen("PLAN.INP","r",stdin);
    freopen("PLAN.OUT","w",stdout);
    cin>>m>>n;
```

```

        for(int i=1;i<=m;i++)
            for(int j=1; j<=n;j++)
                cin>>a[i][j];
        memset(DD,false,sizeof(DD));
    }
    //=====
    void    DFS(int i, int j) {
        DD[i][j] =true ; d++;
        for(int k=0;k<4; k++) {
            int  i1=i+dx[k] ,   j1= j + dy[k] ;
                if(i1>=1&& i1<=m && j1>=1 && j1 <=n && !DD[i1][j1]
&& a[i1][j1]==a[i][j]){
                    DFS(i1,j1);
                }
        }
    }
    //=====
    void    xuly(){
        for(int i=1;i<=m;i++)
            for(int j=1;j<=n;j++)
                if(!DD[i][j])
                {
                    dem++; d=0;
                    DFS(i,j);    //BFS(i,j) ;
                    Max=max(Max,d);
                }
        cout<<dem<<endl;
        cout<<Max;
    }
    //=====
    int main()
    {
        nhap();  xuly();
        return 0;
    }

```

Bài 3: Nông trại (Trích Đề thi HSG lớp 12 năm 2018-2019, tỉnh Hà Tĩnh)

Trong trại chăn nuôi của John có nuôi một số con gà. Trong khi John đang ngủ say, những con cáo đói đã vào trại và tấn công đàn gà.

Trại có dạng hình chữ nhật gồm các ô được đánh số bởi m hàng và n cột. Ký tự dấu chấm '.' là ô trống, ký tự '#' là hàng rào, ký tự 'c' là gà và ký tự 'f' là cáo.

Chúng ta coi 2 ô là cùng một chuồng nếu có thể chuyển từ ô nọ tới ô kia bằng đường đi chỉ gồm các đường đi theo chiều ngang hoặc thẳng đứng không vướng hàng rào. Các ô mà từ chúng có thể thoát khỏi sân không được xem là một phần của bất kì miền nào.

May thay, những con gà biết tự vệ. Chúng có thể mổ chết những con cáo trong chuồng nếu số lượng gà lớn hơn số lượng cáo trong cùng một chuồng. Ngược lại những con cáo sẽ ăn hết các con gà trong chuồng đó.

Ban đầu, các con gà và các con cáo đã được xác định trong các miền của trại. Hãy tính số lượng gà và số lượng cáo còn lại trong sáng hôm sau.

Dữ liệu vào: File văn bản FARM.INP:

- Dòng đầu chứa 2 số nguyên dương m và n ($1 \leq m, n \leq 1000$) là số hàng và số cột của trại.
- m dòng sau, dòng i chứa n ký tự. Ký tự thứ j là ký hiệu của ô (i, j) trong trại.

Kết quả ghi ra file FARM.OUT chỉ một dòng gồm 2 số nguyên cách nhau bởi dấu cách: Số cáo và số gà còn lại trong trại.

Ví dụ:

FARM.INP	FARM.OUT
<pre> 8 8 . # # # # # # . # . . c . . . # # . # # # # . # # . # f . # . # # . # . c # c # # c . # # . . # # . f . . f . # . # # # # # # . </pre>	<pre> 1 3 </pre>

```

#include <bits/stdc++.h>
using namespace std;
int n,m,x,y,ga=0,cao=0;
char a[1003][1003];

```

```

bool DD[1003][1003];
int dx[4]= {0, -1, 0 ,1};
int dy[4]= {-1, 0, 1, 0};
//=====
void nhap()
{
    freopen("FARM.INP","r",stdin);
    //freopen("FARM.OUT","w",stdout);
    cin>>m>>n;
    for(int i=1;i<=m;i++)
        for(int j=1; j<=n;j++)
            cin>>a[i][j];
    memset(DD,false,sizeof(DD));
}
//=====
void DFS(int i, int j) {
    DD[i][j] =true ;
    if(a[i][j]=='c') {ga++; a[i][j]='#';}
    else if(a[i][j]=='f') {cao++; a[i][j]='#';}
    for(int k=0;k<4; k++) {
        int i1=i+dx[k] , j1= j + dy[k] ;
        if(i1>=1&& i1<=m && j1>=1 && j1 <=n && !DD[i1][j1] &&
a[i1][j1]!='#'){
            DFS(i1,j1);
        }
    }
}
//=====
void xuly(){
    int sog=0, socao=0;
    for(int i=1;i<=m;i++)
        for(int j=1;j<=n;j++)
            if(a[i][j]=='c' || a[i][j]=='f')

```

```

        {
            ga=0; cao=0;
            DFS(i,j);
            if(ga>cao) sogas=sogas+ga;
            else socao=socao+cao;
        }
        cout<<socao<<"    "<< sogas;
    }
    //=====
int main()
{
    nhap(); xuly();
    return 0;
}

```

Bài 4: Mê cung

Một mê cung hình chữ nhật kích thước $M \times N$ gồm các ô vuông đơn vị. trên mỗi ô ghi số:

0: nếu ô đó an toàn ; 1: nếu ô đó có chướng ngại

Có một nhà thám hiểm đang đứng tại 1 ô an toàn. Nhà thám hiểm có thể từ 1 ô sang một trong các ô chung cạnh với ô đang đứng. Một cách đi thoát khỏi mê cung là một hành trình đi qua các ô an toàn và ra khỏi mê cung từ 1 ô ở biên.

Yêu cầu: Hãy giúp nhà thám hiểm thoát khỏi mê cung an toàn và nhanh nhất

Dữ liệu vào: MECUNG.INP

Dòng đầu tiên chứa 2 số nguyên M, N ($M, N \leq 1000$)

Dòng thứ 2 chứa 2 số nguyên x, y là vị trí của nhà thám hiểm (luôn an toàn)

M dòng tiếp theo, dòng thứ i chứa N giá trị (1- nếu có chướng ngại, 0 – Không có chướng ngại)

Dữ liệu ra: MECUNG.OUT

Dòng đầu tiên ghi số ô phải đi nếu người đó ra khỏi được mê cung. Ghi 0 nếu người đó không ra khỏi được mê cung. Nếu người đó ra khỏi được thì một số dòng tiếp theo, mỗi số ghi 2 số nguyên x, y lần lượt là vị trí của nhà thám hiểm trong một hành trình ra khỏi mê cung

MECUNG.INP	MECUNG.OUT
------------	------------

6 7	6
4 5	4 5
1 0 0 1 0 1 1	4 4
1 0 1 0 0 1 0	3 4
1 0 0 0 1 0 1	2 4
0 0 1 0 0 1 0	2 5
1 1 0 1 0 1 0	1 5
1 1 0 1 1 0 1	

Ý tưởng thuật toán

- Dùng biến *ktra* để kiểm tra có đường đi hay không
- Trong hàm BFS khi loang ra ô (i1,j1) , nếu ô (i1,j1) là ô ngoài viền của lưới ô vuông không(mê cung) thì biến *ktra=true* và dùng hàm BFS.
- Gọi hàm BFS(x,y) , sau khi kết thúc lời gọi hàm BFS nếu biến *ktra=false* thì đưa ra - 1, ngược lại gọi chương trình con truy vết đường đi. Đưa ra số ô và đường đi.

```
#include <bits/stdc++.h>
using namespace std;
int n,m,x,y,x2,y2,dem=0;
bool ktra=false;
int a[103][103];
int DD[103][103];
int dx[4]= {0, -1, 0 ,1};
int dy[4]= {-1, 0, 1, 0};
int h[4]= {1, 2, 3, 4} ;// Danh dau huong di
//=====
void nhap()
{
    freopen("MECUNG.INP","r",stdin);
    freopen("MECUNG.OUT","w",stdout);
    cin>>m>>n>>x>>y; cout<<m<<" "<<n<<" "<<x<<" "<<y<<endl;
    for(int i=1;i<=m;i++)
        for(int j=1; j<=n;j++)
            cin>>a[i][j];
    memset(DD,0,sizeof(DD));
}
//=====
void BFS(int i, int j) {
    queue<pair<int,int>> q;
    q.push({i, j}) ;
    DD[i][j]=5 ; // danh dau o xuat phat
    while( !q.empty()) {
        pair <int, int> x = q.front() ;
        q.pop();
        for (int k=0; k<4 ; k++) {
            int i1=x.first+dx[k] , j1= x.second + dy[k] ;
```

```

        if(i1>=1&& i1<=m && j1>=1 && j1 <=n && !DD[i1][j1]&&
a[i1][j1]==0)
        {
            q.push({i1, j1});
            DD[i1][j1] = h[k] ;
            if(i1==1||i1==m||j1==1||j1==m)
                { ktra=true; x2=i1;y2=j1; return;
                }
        }
    }
}
//=====
void path(int x1,int y1,int x2,int y2)
{
    vector<pair<int,int>> res ; //dùng vector res để lưu đường đi
    int x = x2 , y = y2 ;
    while( (x != x1) || ( y != y1)){
        res.push_back({x,y}) ;
        for(int i=0;i<4;i++)
            if(DD[x][y]==h[i])
            {
                x = x - dx[i];
                y = y - dy[i];
                break;
            }
    }
    res.push_back({x1,y1});
    reverse(res.begin(), res.end()) ;
    cout<<res.size()<<endl;
    for(auto p:res)
        cout<<p.first<<" "<<p.second<<endl;
}
//=====
void xuly(){
    if(x==1||x==m||y==1||y==n){
        cout<<1<<endl;
        cout<<x<<" "<<y;
        return;
    }
    BFS(x,y);
    if(!ktra) cout<<-1;
    else
    {
        path(x,y,x2,y2);
    }
}
//=====
int main()
{
    nhap();
    xuly();
    return 0;
}

```

2. Một số bài tập tự luyện

Bài 1: Hành trình an toàn (Trích Đề thi HSG lớp 12 năm 2018-2019, tỉnh Hà Tĩnh)

Bản đồ một mê cung là một hình chữ nhật kích thước $m \times n$ được chia thành lưới ô vuông đơn vị trong đó ô nằm trên giao điểm của hàng i và cột j được gọi là ô (i,j) . Trên mỗi ô ghi một trong ba kí tự:

- “.” (dấu chấm): Nếu ô đó an toàn
- “#”: Nếu ô đó không được đi vào
- “*”: Nếu là ô có một nhà thám hiểm đang đứng.

Duy nhất chỉ có một ô ghi dấu “*”. Nhà thám hiểm có thể từ một ô đi sang một trong số các ô chung cạnh với ô đang đứng. Một cách đi thoát khỏi mê cung là một hành trình đi ra một ô biên mà không đi vào ô mang dấu “#”. Hãy chỉ giúp cho nhà thám hiểm một hành trình thoát ra khỏi mê cung. Biết rằng luôn có cách thoát khỏi mê cung đã cho.

Dữ liệu: Vào từ file văn bản LABTH.INP

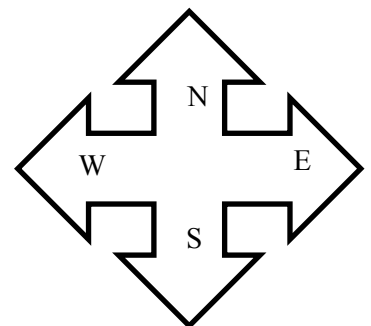
- Dòng 1 chứa hai số nguyên dương $m, n \leq 10^3$;
- M dòng tiếp theo, dòng thứ i chứa n ký tự, ký tự thứ j là ký tự ghi trên ô (i,j) của bản đồ.

Kết quả: Ghi ra file văn bản LABTH.OUT một dãy ký tự liên tiếp. Ký tự thứ k cho biết hướng đi tại bước di chuyển thứ k . Cụ thể là:

- Ký tự “N”: Di chuyển từ một ô sang ô kề cạnh phía trên
- Ký tự “S”: Di chuyển từ một ô sang ô kề cạnh phía dưới
- Ký tự “W”: Di chuyển từ một ô sang ô kề cạnh bên trái
- Ký tự “E”: Di chuyển từ một ô sang ô kề cạnh bên phải

Ví dụ

LABTH.INP	LABTH.OUT
8 8 # . # # # # # # . # * . . . # # . # # # . # # . # . . . # # . # . # # # # # . # . # # # # # # # # # # # # # #	EEESSWWSSSWNNNNNN



Ràng buộc:

- Có 60% số test khác ứng với 60% số điểm của bài có $n \leq 10^2$
- Có 40% số test còn lại ứng với 40% số điểm của bài có $10^2 < n \leq 10^3$

BÀI 2: TRÒ CHƠI CHON QUÀ (Trích Đề thi HSG lớp 12 năm 2021-2022, tỉnh Hà Tĩnh)

Trò chơi được cho bởi một bảng gồm m hàng và n cột, các hàng được đánh số từ 1 đến m (từ trên xuống), các cột được đánh số từ 1 đến n (từ trái sang phải). Ô nằm giao giữa hàng i và cột j là ô (i,j) . Mỗi ô được tô một màu có mã màu là một ký tự thuộc tập $['A', 'B', \dots, 'Z']$, các ô chứa món quà tô màu có mã màu là ký tự '*'. Nhiệm vụ của người chơi là: xuất phát từ một ô (x,y) cần tìm đường đi tới một ô chứa món quà theo quy tắc di chuyển như sau:

Chỉ được di chuyển sang các ô chung cạnh;

Nếu di chuyển sang ô mới cùng mã màu với ô hiện tại thì không mất phí di chuyển, còn nếu di chuyển sang ô mới khác màu với ô hiện tại thì chi phí là.

Yêu cầu: Cho bảng ký tự biểu diễn trò chơi và ô (x,y) , hãy tính chi phí di chuyển của đường đi từ ô (x,y) đến ô chứa ký tự '*'.

Dữ liệu: Vào từ file văn bản TABG.INP

- Dòng đầu chứa ba số nguyên dương m, n, Q ;
- m dòng tiếp theo, mỗi dòng chứa một xâu độ dài n mô tả bảng ký tự là mã màu của các ô;
- Q dòng tiếp theo, mỗi dòng chứa hai số nguyên x, y .

Kết quả: Ghi ra file văn bản TABG.OUT :

- Gồm Q dòng, mỗi dòng ghi một số nguyên là chi phí di chuyển của đường đi từ ô (x,y) đến ô chứa ký tự '*'.

Các số ghi trên một dòng của input/output files được/phải ghi cách nhau ít nhất một dấu cách.

Ràng buộc:

- Có 40% số test tương ứng với 40% số điểm thỏa mãn điều kiện $m, n \leq 10^2$; $Q=1$;
- Có 40% số test khác tương ứng với 40% số điểm thỏa mãn điều kiện $m, n \leq 10^3$; $Q \leq 3$;
- Có 20% số test khác tương ứng với 20% số điểm thỏa mãn điều kiện $m, n \leq 10^3$; $Q \leq m \times n$;

Ví dụ:

TABG.INP	TABG.OUT
4 6 3	1
*ADFEB	2
AD*CFB	3
ACBADA	
AAAAAA	
3 6	
1 6	
1 5	

Bài 3: Diện tích REGION (Trích Đề thi HSG lớp 12 năm 2016-2017, tỉnh Hà Tĩnh)

Trong một lưới ô vuông mà mỗi ô vuông chứa một số 0 hoặc 1, dãy các ô vuông chứa số 1 mà hai ô liên tiếp có chung cạnh hoặc đỉnh và ô cuối cùng của dãy có chung cạnh hoặc đỉnh với ô đầu tiên tạo thành một đường kín bọc được một vùng ô mà ta gọi là một Region. Diện tích của một Region là số ô vuông chứa số 0 nằm trong đó.

Cho trước một lưới ô vuông kích thước $M \times N$ chứa duy nhất 1 Region, hãy tính diện tích của Region đó.

Dữ liệu vào từ tệp văn bản REGION.INP có cấu trúc:

- Dòng đầu tiên ghi hai số nguyên dương M, N ($5 \leq M, N \leq 1000$) cách nhau ít nhất một dấu cách.
- M dòng tiếp theo mô tả lưới ô vuông cho trước, mỗi dòng chứa dãy gồm N số 0 hoặc 1 được ghi liền nhau.

Dữ liệu ra ghi vào tệp văn bản REGION.OUT một số nguyên là diện tích của Region trên lưới ô vuông đã cho.

Ví dụ:

REGION.INP	REGION.OUT
6 8 01000000 10100000 10010000 10001000 01010000	7

Ràng buộc:

- Có 50% số test ứng với 50% số điểm của bài có $5 \leq M, N \leq 100$.
- Có 50% số test còn lại ứng với 50% số điểm của bài có $M, N \leq 1000$.

Bài 4 Thỏ và Cà rốt (*Trích đề thi HSG lớp 12 Tỉnh Hà Tĩnh năm 2014 - 2015*)

Giải toánn Violympic là một sân chơi hữu ích cho các em học sinh, nhất là các em học sinh ở bậc tiểu học và THCS. Đối với cấp THCS mỗi vòng thi thường có trò chơi Thỏ và Cà rốt. Trò chơi được thực hiện trên một bảng ô vuông góc trái trên và củ Cà rốt ở góc phải dưới của bảng, một số ô vuông trong bảng có chứa vật cản ướng với một câu hỏi toán học. Nhiệm vụ của thí sinh là di chuyển con thỏ tới ô vuông có củ Cà rốt. Từ ô vuông đang đứng con Thỏ có thể di chuyển được sang ô vuông kề cách với nó. Trong quá trình di chuyển con Thỏ có thể qua một số ô vuông có vật cản. Để di chuyển qua được ô vuông có vật cản, co Thỏ phải trả lời đúng câu hỏi toán học trong ô vuông đó, ngược lại không thể di chuyển qua được. Để dành thời gian cho bài thi sau, các thí sinh muốn đưa con thỏ của mình tới được ô vuông có củ Cà rốt càng nhanh càng tốt nên luôn tìm đường đi để di chuyển con Thỏ qua ít ô vuông có vật cản nhất.

Bài toán được đặt ra là: Cho một bảng gồm các ô vuông và N câu hỏi được đặt trong N ô vuông nào đó của bảng, các ô vuông được đánh số lần lượt từ trái qua phải từ trên xuống dưới. Con Thỏ được đặt ở ô vuông từ góc trái trên của bảng và giả sử ó trả lời đúng các câu hỏi toán học trong các ô vuông mà nó đi qua. Còn củ Cà rốt được đặt ở ô vuông bất kỳ trong bảng. Tại ô vuông có củ Cà rốt thì không có câu hỏi nhưng tại ô vuông có con Thỏ có thể sẽ có câu hỏi.

Hãy tìm đường đi cho con Thỏ sao cho từ vị trí ban đầu đi tới ô vuông có củ Cà Rốt gặp ít câu hỏi toán học nhất.

Dữ liệu vào là tệp văn bản RABBIT.INP có cấu trúc:

- Dòng thứ nhất chứa 3 số nguyên dương N, x và y với (x,y) là tọa độ vị trí ô vuông đặt củ Cà rốt ($1 \leq N \leq 5 \times 10^4$, $1 \leq x, y \leq 10^3$)

- N dòng tiếp theo, mỗi dòng chứa 2 số (u,v) là tọa độ của vị trí ô vuông có câu hỏi toán học ($1 \leq u, v \leq 10^3$).
- Các số trên một dòng cách nhau ít nhất một dấu cách.

Dữ liệu ra ghi vào tệp văn bản RABBIT.OUT một số nguyên duy nhất là số lượng câu hỏi tối thiểu cần trả lời trên đường đi khi di chuyển chú thỏ tới củ Cà rốt.

Ví dụ:

RABBIT.INP	RABBIT.OUT
7 6 3 6 2 5 2 4 3 2 1 7 3 5 4 6 4	1

Bài 5: Gặp gỡ của hai Robot.

Trên một lưới ô vuông $M \times N$ ($M, N < 100$), người ta đặt Robot A ở góc trái trên, Robot B ở góc phải dưới. Mỗi ô của lưới ô có thể đặt một vật cản hoặc không (ô trái trên và ô phải dưới không có vật cản). Hai Robot bắt đầu di chuyển đồng thời với tốc độ như nhau và không Robot nào được dừng lại trong khi Robot kia di chuyển (trừ khi nó không thể đi được nữa). Tại mỗi bước, Robot chỉ có thể di chuyển theo 4 hướng - đi lên, đi xuống, sang trái, sang phải - vào các ô kề cạnh. Hai Robot gặp nhau nếu chúng cùng đứng trong một ô vuông. Bài toán đặt ra là tìm cách di chuyển ít nhất mà 2 Robot phải thực hiện để có thể gặp nhau.

Dữ liệu vào cho bởi tệp robot.inp:

- Dòng đầu ghi 2 số M, N cách nhau ít nhất một ký tự trống;
- M dòng tiếp theo, mỗi dòng ghi N số 0 hoặc 1 liên tiếp nhau mô tả trạng thái của các ô vuông: 1 - có vật cản, 0 - không có vật cản.

Dữ liệu ra ghi vào tệp robot.out:

- Nếu 2 Robot không thể gặp nhau thì ghi ký tự '#'.

- Ngược lại, ghi hai dòng, mỗi dòng là một dãy các ký tự viết liền nhau mô tả các bước đi của Robot: U - đi lên, D - đi xuống, L - sang trái, R - sang phải. Dòng đầu là các bước đi của Robot A, dòng sau là các bước đi của Robot B.

Ví dụ:

ROBOT.INP	ROBOT.OUT	ROBOT.INP	ROBOT.OUT
4 6	DRRR	3 4	#
011000	LULU	0000	
000001		0000	
001001		0000	
010100			

Bài 6: Trò chơi ô băng

Trên một lưới kích thước $N \times M$ có một người chơi đang đứng. Ô (i,j) đại diện cho ô vuông ở hàng thứ i từ phía trên và cột thứ j từ bên trái của lưới này. Mỗi ô vuông của lưới này là băng hoặc đá, được biểu diễn bằng N xâu $S_1, S_2, \dots, S_N$, có độ dài M như sau:

- Nếu ký tự thứ j của S_i là ".", ô vuông (i,j) là băng.
- Nếu ký tự thứ j của S_i là "#", ô vuông (i,j) là đá.

Viên bên ngoài của lưới này (tất cả các ô vuông ở hàng thứ 11, hàng thứ N , cột thứ 11, cột thứ M đều là đá.

Ban đầu, người chơi đứng trên ô vuông $(2,2)$, là một ô băng. Người chơi có thể thực hiện các bước di chuyển theo các hướng sau:

- Xác định hướng di chuyển: lên, xuống, trái hoặc phải.
- Tiếp tục di chuyển theo hướng đã chọn cho đến khi người chơi va vào một ô vuông là đá.

Cụ thể, thực hiện các bước sau:

- Nếu ô vuông tiếp theo trong hướng di chuyển là băng, di chuyển đến ô đó và tiếp tục di chuyển.

- Nếu ô vuông tiếp theo trong hướng di chuyển là đá, ở lại ô hiện tại và dừng di chuyển.

Hãy tìm số lượng ô vuông bằng mà người chơi có thể tiếp xúc (đi qua hoặc dừng lại).

Dữ liệu vào: Tập OBANG.INP

- Dòng đầu là 2 số nguyên N, M ($3 \leq N, M \leq 200$)
- N dòng tiếp, mỗi dòng là một xâu kí tự S_i có độ dài M gồm các ký tự "#" và ".".
- Ô vuông (i, j) là đá nếu $i=1, i=N, j=1$ hoặc $j=M$
- Ô vuông $(2, 2)$ là bằng.

Dữ liệu ra: Tập OBANG.OUT chứa số duy nhất là Số ô bằng mà người chơi có thể tiếp xúc

Ví dụ:

OBANG.INP	OBANG.OUT
<pre> 6 6 ##### #...# #.#..# #..#.# #....# ##### </pre>	<pre> 12 </pre>

Bài 7:

<https://www.spoj.com/PTIT/problems/BCROBOT/cstart=10>

Bài 8:

<https://www.spoj.com/PTIT/problems/BCLKCOUN/>